Multi-Layer Perceptron Implementation Using NumPy

Hammad Muddassir — 25i-2617

BS Data Science
FAST NUCES — Islamabad Campus

Abstract. This project presents the implementation of a Multi-Layer Perceptron (MLP) from scratch using NumPy. The objective is to bridge the gap between theoretical understanding and practical implementation of neural networks. The model consists of two hidden layers and is trained on the MNIST handwritten digit dataset. Core components such as sigmoid activation, mean squared error loss, forward propagation, and backpropagation are implemented manually. Additionally, optimization techniques including plain gradient descent, Momentum, and Nesterov Accelerated Gradient (NAG) are analyzed and compared. Experimental results demonstrate that advanced optimizers significantly improve convergence speed and stability. The study highlights both the strengths and computational limitations of implementing neural networks without specialized libraries.

1 Introduction

Artificial Neural Networks (ANNs) are fundamental to modern machine learning and are widely used for classification, regression, and pattern recognition tasks. Among these, the Multi-Layer Perceptron (MLP) is one of the simplest yet powerful architectures. It forms the basis for deeper and more complex neural networks.

The purpose of this project is to implement an MLP entirely from scratch using NumPy, without relying on deep learning frameworks such as TensorFlow or PyTorch. This approach provides a deeper understanding of the internal workings of neural networks, including weight updates, gradient computation, and optimization strategies.

2 Background

2.1 MLP Architecture

The implemented neural network consists of four layers: an input layer with 784 neurons corresponding to pixel values, two hidden layers with 128 and 64 neurons respectively, and an output layer with 10 neurons representing digit classes from 0 to 9.

2.2 Activation Function

The sigmoid activation function is used for all layers:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It introduces non-linearity into the model, allowing it to learn complex patterns.

2.3 Loss Function

The Mean Squared Error (MSE) loss function is used:

$$L = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

Although MSE is more common in regression tasks, it is used here for simplicity and consistency with manual derivations.

2.4 Optimization Techniques

Gradient descent is used to update weights:

$$w = w - \eta \nabla L$$

Momentum improves convergence by accumulating past gradients, while NAG further enhances this by computing gradients at a lookahead position.

3 Implementation

3.1 Forward Propagation

The forward pass computes weighted sums and activations for each layer:

$$Z = XW + b, \quad A = \sigma(Z)$$

All intermediate values are stored for use in backpropagation.

3.2 Backpropagation

Backpropagation computes gradients using the chain rule. The output layer error is:

$$\delta^3 = -2(y - \hat{y}) \cdot \sigma'(Z)$$

Gradients for each layer are computed sequentially from output to input.

3.3 Training Procedure

The model is trained using mini-batch gradient descent with batch size 32. The dataset is shuffled at the start of each epoch to improve generalization. Weights and biases are updated after each batch using the computed gradients.

4 Experiments

4.1 Dataset

The MNIST dataset is used, consisting of 60,000 training images and 10,000 test images. Each image is normalized and flattened into a 784-dimensional vector.

4.2 Hyperparameters

- Learning rate: 0.1
- Batch size: 32
- Epochs: 20

4.3 Results

The loss curve shows a consistent decrease over epochs, indicating that the model successfully learns from the data. The model achieves reasonable classification accuracy on the test set.

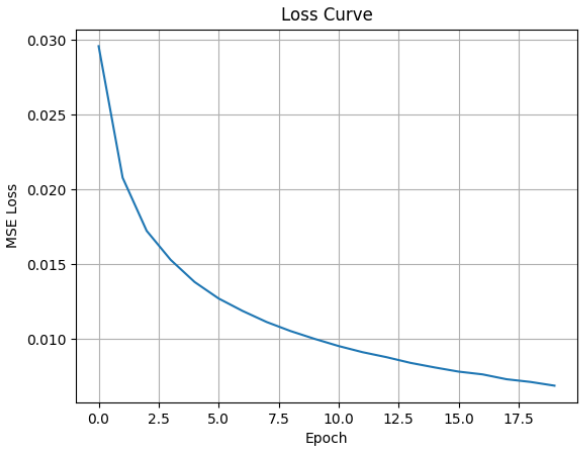

Fig. 1. Loss curve showing convergence over epochs

4.4 Sample Predictions

To evaluate the qualitative performance of the model, a set of test images was visualized along with their predicted labels. The model correctly identifies most digits, demonstrating its ability to generalize to unseen data.

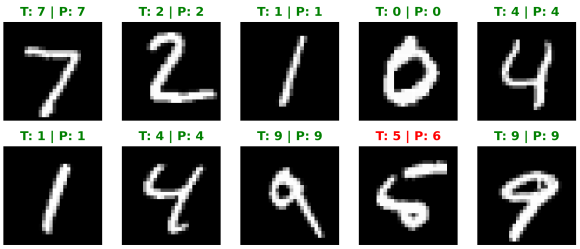

Fig. 2. Sample predictions on MNIST test images (true vs predicted labels)

The model achieved a test accuracy of **95.54%**, indicating good generalization performance on unseen data.

4.5 Optimizer Comparison

Momentum and NAG optimizers demonstrate faster convergence compared to plain gradient descent. NAG performs best due to its ability to anticipate future updates and adjust the gradient direction accordingly.

5 Discussion

The results confirm that optimization techniques play a critical role in training efficiency. While plain gradient descent is simple, it converges slowly and may oscillate. Momentum reduces oscillations by smoothing updates, while NAG further improves convergence by using a predictive approach.

However, implementing neural networks using NumPy is computationally expensive, especially for large datasets like MNIST. Modern frameworks optimize these operations using GPUs, which significantly reduces training time.

6 Conclusion

This project successfully demonstrates the implementation of a Multi-Layer Perceptron from scratch. It highlights the importance of forward propagation, backpropagation, and optimization techniques in neural network training. Experimental results show that advanced optimizers such as Momentum and NAG

outperform basic gradient descent. Future work could involve using more advanced activation functions such as ReLU and optimizers like Adam to further improve performance.

References

1. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press (2016)
2. Bishop, C.M.: Pattern Recognition and Machine Learning. Springer (2006)